

TECHNICAL DOCUMENTATION

Design & Build a RAG-Based Customer Support Assistant

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines a retrieval system with a language model. Instead of relying only on what the model learned during training, RAG first searches an external document store for relevant content, then uses that content to generate the answer. This produces accurate, grounded, up-to-date responses.

Why is it needed?

- LLMs hallucinate when uncertain - RAG grounds answers in real documents
- Training data has a cutoff - company documents are never in it
- Knowledge updates need only a document update, not model retraining
- Reduces wrong answers reaching customers significantly

Use case overview:

A company uploads its FAQ and policy PDFs. Customers ask questions. The system retrieves the most relevant policy text and returns a direct answer. Complex queries are escalated to a human support agent via HITL.

2. System Architecture Explanation

Two-Phase Architecture:

Phase 1 - Ingestion (run once):

- Load PDF using PyPDFLoader
- Split into 300-character chunks with 50-character overlap
- Generate embeddings using HuggingFace MiniLM model
- Store all vectors in ChromaDB

Phase 2 - Query (per request):

- User submits natural language query
- LangGraph wraps query in GraphState
- Retriever fetches top-1 most similar chunk from ChromaDB
- Process node checks answer confidence
- High confidence: return answer to user
- Low confidence: route to HITL escalation node

3. Design Decisions

Chunk size: 300 characters, overlap: 50

300 characters fits a typical FAQ answer (1-3 sentences) into one chunk. 50-character overlap preserves sentence continuity across boundaries. Too large: retrieval becomes imprecise. Too small: context is split.

Embedding model: sentence-transformers/all-MiniLM-L6-v2

Chosen because it is free (no API key required), runs locally (no network latency), and produces strong semantic similarity on standard benchmarks. Output is 384 dimensions - compact and fast to search.

Retrieval: top-1 similarity search (k=1)

Single most relevant chunk is returned. For FAQ use case, one focused answer is cleaner than blending multiple chunks. For complex domains, k=3 with re-ranking would be appropriate.

Prompt design:

Retrieved chunk is returned directly to avoid requiring an API key. In production, a prompt template would wrap context: 'Given context: {context}, answer: {question}'.

4. Workflow Explanation (LangGraph)

Node responsibilities:

- process_node -> queries ChromaDB, writes answer into state
- hitl_node -> replaces answer with escalation message, logs ticket

State transitions:

```
Initial: { question: 'What is return policy?', answer: '' }
|
v process_node
{ answer: 'Items can be returned within 30 days...' }
-> len >= 50 -> END (return answer)
```

```
Initial: { question: 'Can I sue?', answer: '' }
|
v process_node
{ answer: 'Legal.' } <- only 6 chars
-> len < 50 -> hitl_node
{ answer: 'Escalated to human support.' }
```

5. Conditional Logic

Routing decisions:

- len(answer) >= 50 -> direct answer, high confidence
- len(answer) < 50 -> HITL escalation, low confidence
- Future: add keyword blacklist ('legal', 'complaint') -> always escalate
- Future: use ChromaDB distance score as confidence measure

6. HITL Implementation

Role of human intervention:

HITL is a designed part of the workflow, not just a fallback. Some queries need human judgement: legal matters, personalised requests, emotionally sensitive situations, or queries needing live account access. The system handles ~80 percent of routine queries automatically.

Benefits:

- Prevents wrong answers from reaching customers
- Builds trust by transparently acknowledging limitations
- Human-approved answers can improve the knowledge base over time

Limitations:

- Escalation adds delay compared to instant AI response
- Requires a human support team to handle escalated tickets
- Current threshold ($len < 50$) is a rough heuristic needing calibration

7. Challenges and Trade-offs

- Retrieval accuracy vs speed -> $k=1$ for FAQ; $k=3$ for technical docs
- Chunk size vs context quality -> 300 chars; tune per domain
- Local vs API embedding -> Local MiniLM for cost and privacy
- Confidence threshold -> $len < 50$ as start; calibrate on real traffic
- LLM cost vs quality -> GPT-3.5 for production; GPT-4 for review

8. Testing Strategy

Unit tests:

- Chunker: verify chunk count, size constraints, overlap
- Retriever: given known query, assert correct chunk is top result
- Routing: $len < 50$ triggers HITL, $len \geq 50$ returns direct answer
- HITL: verify ticket logged with timestamp and query

Sample queries:

- 'What is your refund policy?' -> Direct answer
- 'How do I contact support?' -> Direct answer
- 'xyz123 ???' -> HITL escalation
- 'What are your shipping options?' -> Direct answer
- 'Can I get a refund after 60 days?' -> Direct or HITL

9. Future Enhancements

Near-term:

- Multi-document support: load entire folders of PDFs
- Persistent ChromaDB: save vector store to disk across sessions

- Web UI: FastAPI + React interface for real customer use
- Improved confidence: use ChromaDB distance scores

Long-term:

- LangGraph multi-agent: separate agents for billing, shipping, technical
- Feedback loop: human-approved answers written back to ChromaDB
- Multilingual support using multilingual-MiniLM model
- Production deployment with Docker on cloud (GCP/AWS)

10. Conclusion

This RAG-based customer support assistant shows how LangChain, LangGraph, ChromaDB, and HuggingFace can be combined into a working AI system in under 100 lines of Python. The system separates ingestion from retrieval, makes routing explicit and inspectable, and handles failures gracefully through HITL.

The most important insight: the hard part of RAG is not the code but the decisions - chunk size, k, confidence threshold, and escalation criteria. Each requires domain knowledge and iteration on real traffic.